

Machine Language Programming

Lab 4 — Nand2Tetris Project 4 (Hack Assembly)

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 21, 2026

Aim. *To learn the Hack instruction set architecture — the A-instruction and C-instruction formats — and to hand-translate, write, and test low-level assembly programs that run on the CPU Emulator.*

4.1 Learning Outcomes

On completing this lab, a student will be able to:

- Distinguish A-instructions (`@value`) from C-instructions (`dest=comp;jump`) in the Hack ISA.
- Hand-translate a short assembly program into its 16-bit binary machine code.
- Write iterative assembly programs using loops, conditional jumps and memory-mapped I/O.
- Use the memory map (RAM, Screen at 16384, Keyboard at 24576) to interact with peripherals from assembly.

4.2 Pre-Lab Reading

Pre-Lab Reading. This lab’s ISA prerequisites were introduced in **Week 4**: instruction formats and operand types (Mano, *Computer System Architecture*, 3rd ed., Sec. 8-4, p.255, and Table 8-1’s transfer-encoding logic, p.243), and the instruction cycle (Hamacher Ch. 7, chapter-level). Review that lecture’s treatment of how an instruction’s bit pattern is decoded into a register-transfer operation — the Hack C-instruction’s `dest=comp;jump` fields are a concrete, minimal instance of exactly that idea: a small opcode-like field selects a computation, and separate fields select where the result is stored and whether a jump is taken.

What this lab adds that Week 4 does not cover: the Hack ISA’s specific bit layout (A-instruction: 0 + 15-bit constant; C-instruction: 111 a ccccc ddd jjj) and its memory map (data RAM 0--16383, memory-mapped Screen 16384--24575, memory-mapped Keyboard at 24576) — see Nisan & Schocken, *The Elements of Computing Systems*, Ch. 4, *Machine Language*, and the Hack instruction-set reference table therein (comp/dest/jump mnemonics and their bit encodings).

4.3 Toolchain Setup

1. Install a Java Runtime Environment (JRE), version 8 or later, if not already present.
2. Download the Nand2Tetris software suite (free, nand2tetris.org) — this lab introduces the **CPU Emulator** and the **Assembler**, in addition to the Hardware Simulator used in Labs 1-3.
3. The Assembler translates a `.asm` text file into a `.hack` binary file (one 16-bit instruction per line, as 0/1 characters) — use it to check your hand-assembled binary against the tool’s output.
4. The CPU Emulator loads a `.asm` or `.hack` program, lets you single-step or run it, and displays RAM/Screen/Keyboard state live — this is how `Fill.asm` is demonstrated interactively (there is no scripted `.tst` for keyboard input).
5. For `Mult.asm`, load the supplied `Mult.tst` script in the CPU Emulator (FILE → LOAD SCRIPT) and run it — it pre-loads `RAM[0]` and `RAM[1]`, runs your program, and checks `RAM[2]` against `Mult.cmp`.

4.4 Tasks

1. **Hand-assembly exercise.** By hand, translate the 4-line sample program below into 16-bit binary machine code, then verify your answer against the Assembler’s output.

```
@2
D=A
@3
D=D+A
```

Table 1 gives the field-by-field derivation for the first two lines — complete the remaining two yourself.

2. `Mult.asm` — multiply `RAM[0] × RAM[1]` using repeated addition, and store the result in `RAM[2]`. (There is no `MUL` instruction in Hack — the algorithm is: initialize a result to 0, then loop, adding `RAM[0]` to the result and decrementing a counter initialized to `RAM[1]`, until the counter reaches 0.)
3. `Fill.asm` — continuously poll the keyboard and blacken/whiten the entire screen while any key is / is not pressed, using an event loop over the screen’s memory-mapped words (256 rows × 512 columns = 8192 16-bit words, addresses 16384 through 24575).
4. Load and single-step both programs in the CPU Emulator; verify `Mult.asm` against `Mult.cmp` and demonstrate `Fill.asm` interactively (press and release a key, observe the screen toggle).
5. Note in the journal how many machine instructions each program compiles to.

Assembly line	Instruction type & fields	16-bit binary
@2	A-instruction: opcode 0 + value 2	0000000000000010
D=A	C-instruction: comp =A (110000), dest =D (010), no jump (000)	1110110000010000
@3	(complete this row)	
D=D+A	(complete this row)	

Table 1: Hand-assembly worked example (first two lines) — fill in the remaining two rows yourself before checking against the Assembler.

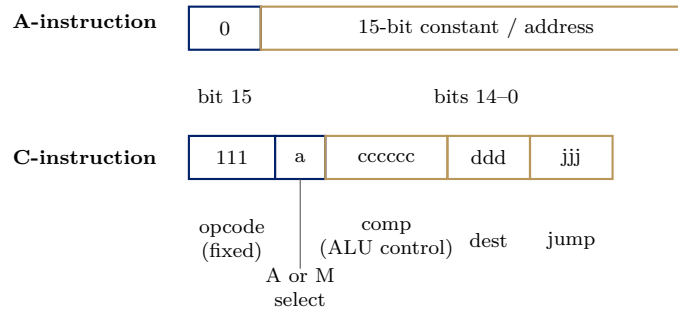


Figure 1: The two Hack instruction formats. A-instruction: opcode bit 0 plus a 15-bit constant loaded into the A-register. C-instruction: opcode bits 111, an **a** bit selecting the ALU’s *y* input as A or M, six **comp** bits (this is literally Lab 2’s ALU control-bit table, minus **zx/no** folded in), three **dest** bits and three **jump** bits.

4.5 Figures

4.6 Instruction Encoding Tables (the ISA “truth tables”)

Decoding a 16-bit instruction step by step:

1. If `instruction[15]=0` it is an A-instruction: load `instruction[14..0]` into the A-register.
2. If `instruction[15..13]=111` it is a C-instruction: the **a** bit (`instruction[12]`) selects the ALU’s *y* input as *A* or *M*; `instruction[11..6]` (`cccccc`) is the **comp** field; `instruction[5..3]` (`ddd`) the **dest** field; `instruction[2..0]` (`jjj`) the **jump** field.
3. $\text{loadA} = \overline{\text{instruction}[15]} + d_1$, $\text{loadD} = \text{instruction}[15] \cdot d_2$, $\text{writeM} = \text{instruction}[15] \cdot d_3$; $\text{jump_taken} = \text{instruction}[15] \cdot [(j_{GT} \cdot \overline{zr} \cdot \overline{ng}) + (j_{EQ} \cdot zr) + (j_{LT} \cdot ng)]$.

4.7 Observation / Journal Table

Journal instruction (separate from the table above): record the completed hand-assembled binary code for the sample program (Table 1, all 4 rows) in the lab journal. This is the syllabus’s own Deliverable 3.

4.8 Viva-Voce Questions

1. Given a C-instruction’s bit pattern, decode it by hand into its **comp/dest/jump** mnemonic form (an instructor-supplied pattern will be given at the viva).

<i>a</i> = 0		<i>a</i> = 1	
comp	<i>cccccc</i>	comp	<i>cccccc</i>
0	101010	M	110000
1	111111	!M	110001
-1	111010	-M	110011
D	001100	M+1	110111
A	110000	M-1	110010
!D	001101	D+M	000010
!A	110001	D-M	010011
-D	001111	M-D	000111
-A	110011	D&M	000000
D+1	011111	D M	010101
A+1	110111		
D-1	001110		
A-1	110010		
D+A	000010		
D-A	010011		
A-D	000111		
D&A	000000		
D A	010101		

Table 2: The `comp` field: all 28 ALU computations and their *cccccc* encoding (the *a* bit is `instruction[12]`, separate from *cccccc*). The six bits are wired directly to `zx,nx,zy,ny,f,no`.

2. Why does the Hack ISA have no dedicated multiply instruction, and what algorithmic consequence does that have for `Mult.asm`'s running time as `RAM[1]` grows?
3. Explain how `Fill.asm` uses the memory-mapped Screen region — why does writing to `RAM[16384]` change pixels rather than ordinary data?
4. What is the `a` bit in a C-instruction for, and how does it let one `comp` field serve both register-only and memory-referencing computations?
5. Why must `Mult.asm` initialize its result register to 0 explicitly, rather than relying on `RAM[2]`'s power-on value?

4.9 Rubric

Reproduced verbatim from `syllabi/CS401_Lab_Syllabus_Nand2Tetris.pdf`.

dest mnemonic	<i>ddd</i>	jump mnemonic	<i>jjj</i>
(none)	000	(none)	000
M	001	JGT	001
D	010	JEQ	010
MD	011	JGE	011
A	100	JLT	100
AM	101	JNE	101
AD	110	JLE	110
AMD	111	JMP	111

Table 3: The `dest` field ($d_1=A$, $d_2=D$, $d_3=M$) and `jump` field ($j_{GT}=\text{instruction}[0]$, $j_{EQ}=\text{instruction}[1]$, $j_{LT}=\text{instruction}[2]$). Note the jump bits are stored *reversed* relative to their reading order: JGT is the least-significant bit.

Program	# machine instructions compiled	Pass / Demo confirmed
Hand-assembly exercise (4 lines)		
<code>Mult.asm</code> (vs. <code>Mult.cmp</code>)		
<code>Fill.asm</code> (interactive demo)		

Table 4: Lab 4 observation table.

Criterion	Marks
<code>Mult.asm</code> passes <code>Mult.cmp</code>	4
<code>Fill.asm</code> correctly toggles the full screen on key press/release	3
Hand-assembly exercise correct	2
Viva: can decode a given C-instruction bit pattern by hand	1
Total	10